



大模型应用技术



前言

- 本章主要介绍大模型应用技术的发展以及大模型的工具调用、包括 Prompt、RAG、Skill、MCP等。



课程目标

- 学完本课程后，您将能够：
 - 了解大模型应用技术的发展；
 - 掌握提示词工程，学会如何编写提示词；
 - 掌握大模型的局限性及大模型如何调用工具
 - 掌握RAG的基本概念，并理解RAG架构设计；
 - 掌握MCP及Function calling的基本概念及交互逻辑
 - 掌握Skills的基本概念及项目结构



目录

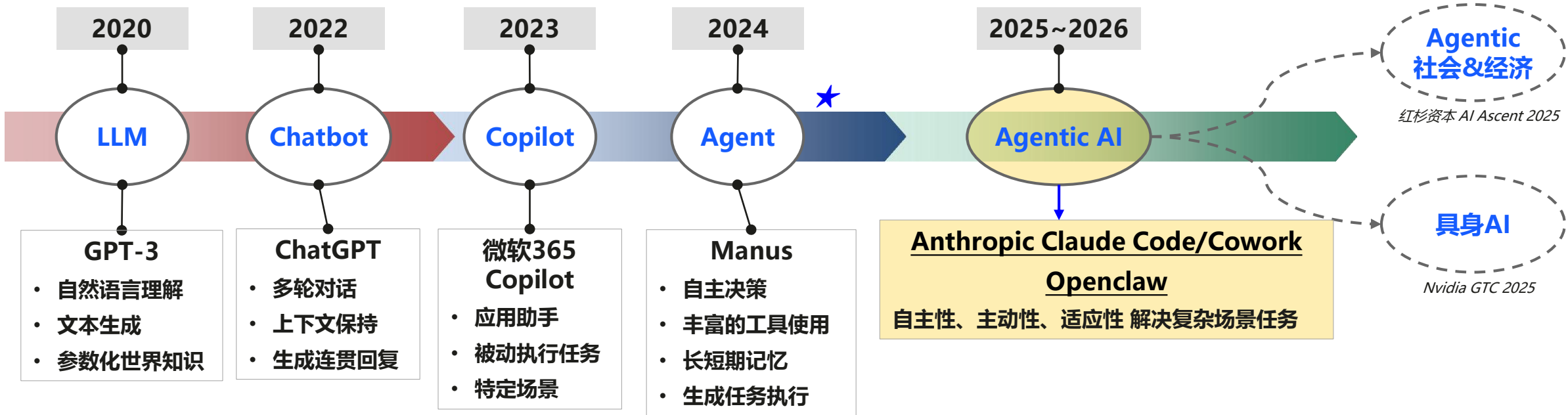
1. 大模型应用技术发展

2. Prompt工程

3. 大模型的局限与工具调用

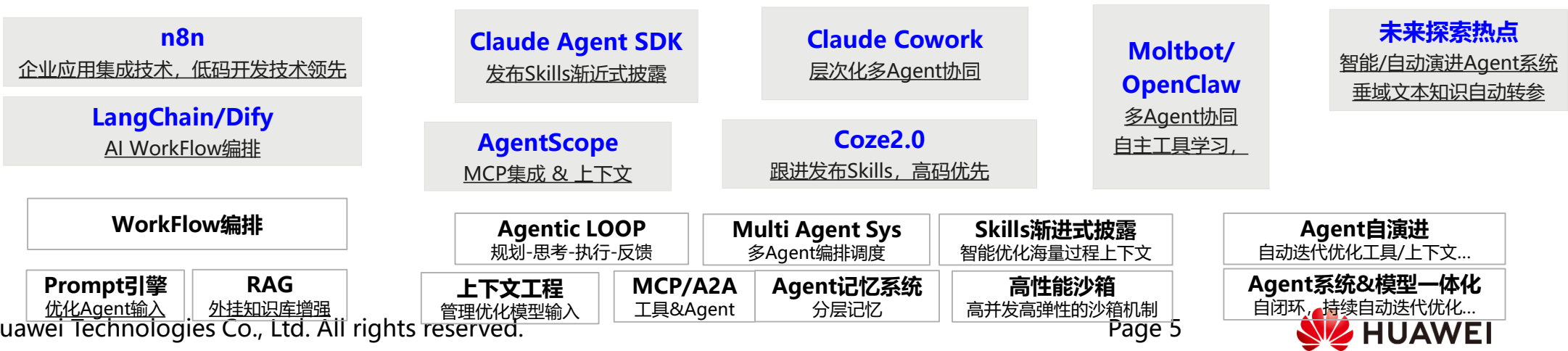
- 2.1 RAG
- 2.2 Plugin与Tools
- 2.3 Function Calling与MCP
- 2.4 Skills

从辅助顾问到个人助手再到自迭代演进的超人



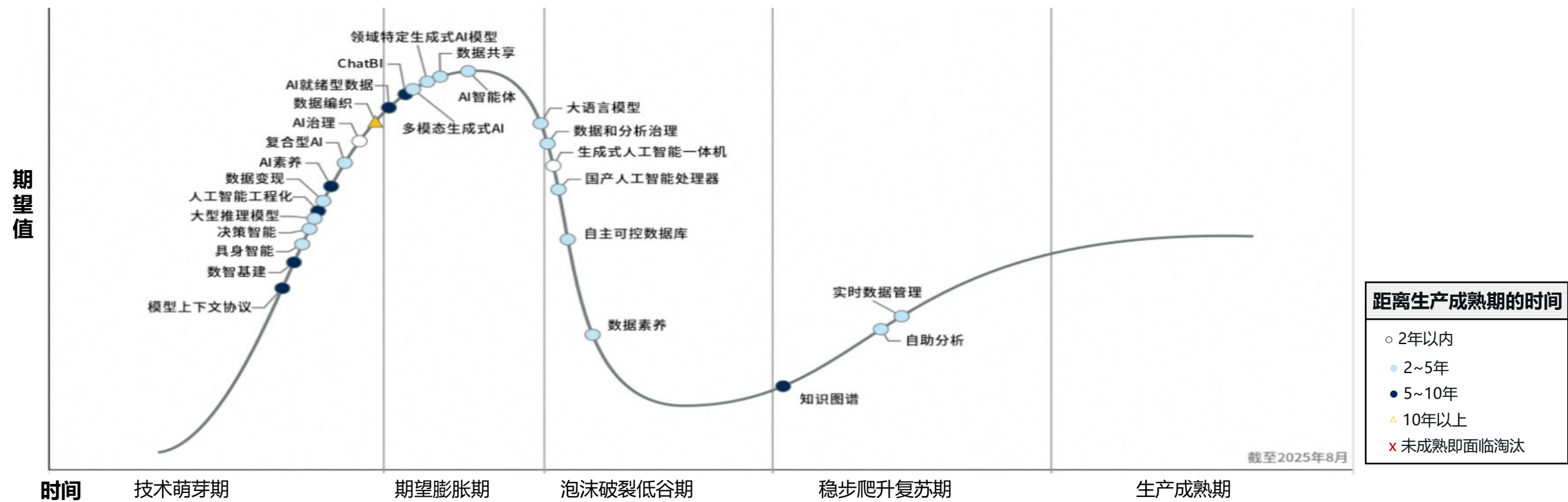
Agent 产品和平台

热点技术

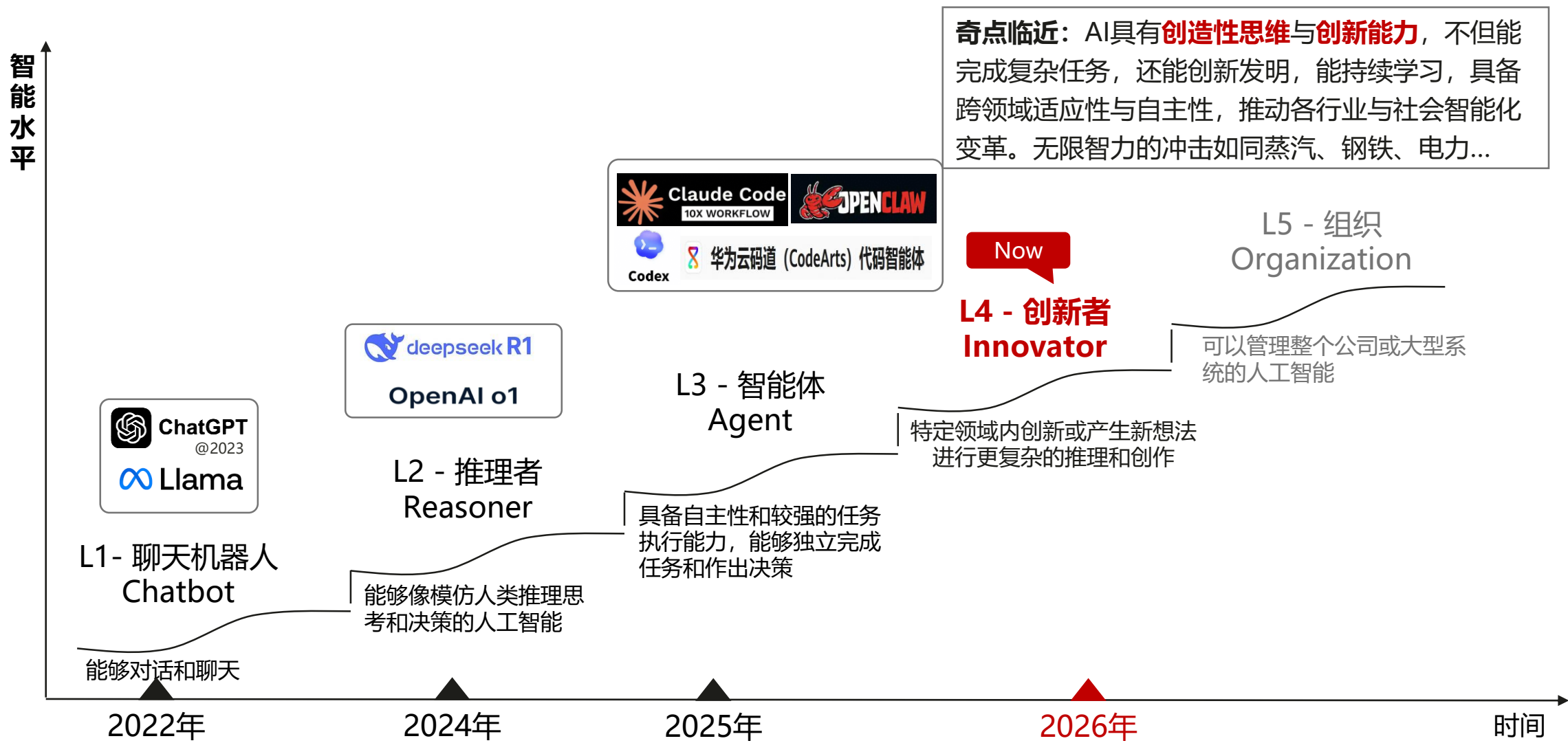


大模型应用日趋成熟，AI应用已处于爆发期

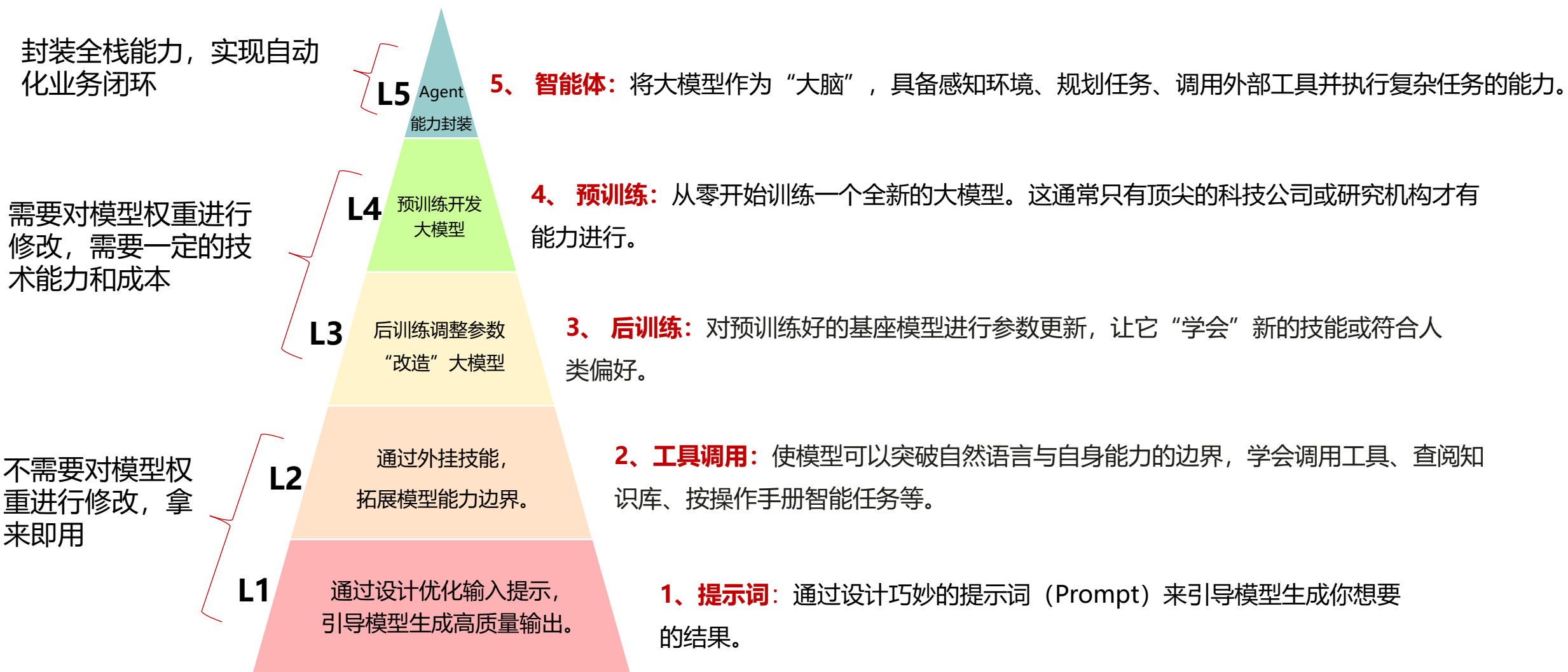
商业与技术洞察公司Gartner于近日发布最新2025年中国数据、分析和人工智能技术成熟度曲线，生成式AI和代理型AI是本技术成熟度曲线的两大核心主题，共同塑造了2025年中国AI发展的主要趋势。大模型应用为代表的AI智能体预计2-5年内将达到生产成熟。



大模型应用将对社会产生革命性影响



大模型应用落地的五级阶梯





目录

1. 大模型应用技术发展

2. Prompt工程

3. 大模型的局限与工具调用

- 2.1 RAG
- 2.2 Plugin与Tools
- 2.3 Function Calling与MCP
- 2.4 Skills

大模型工程应用落地的五级阶梯



1、提示词：提示词工程无需对模型参数进行更新，而是通过设计合理的提示，激发预训练模型中蕴含的知识，从而在下游任务中获得良好的生成质量。

人类如何使能大模型

提示词（Prompt）是决定输出质量与用户体验的关键因素，可帮助大模型明确任务目标、规范输出格式、控制内容风格、实现个性化响应。



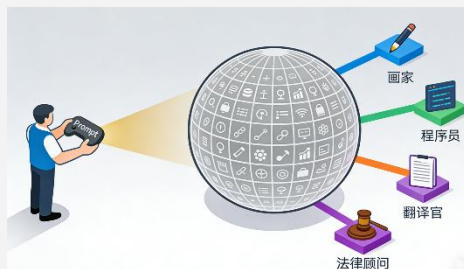
人类

Prompt



模型

核心前提：大模型能力需要“激活”



大模型需要人类的有效引导和赋能

核心逻辑：需求→传递→输出



明确需求

梳理自身
核心诉求



传递意图

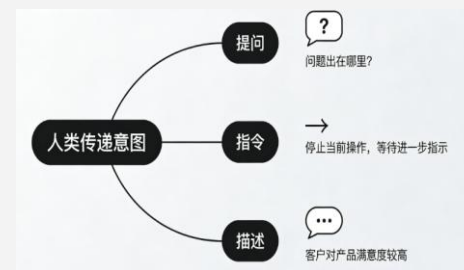
建立与大模型
的认知连接



引导输出

让能力匹配
具体应用场景

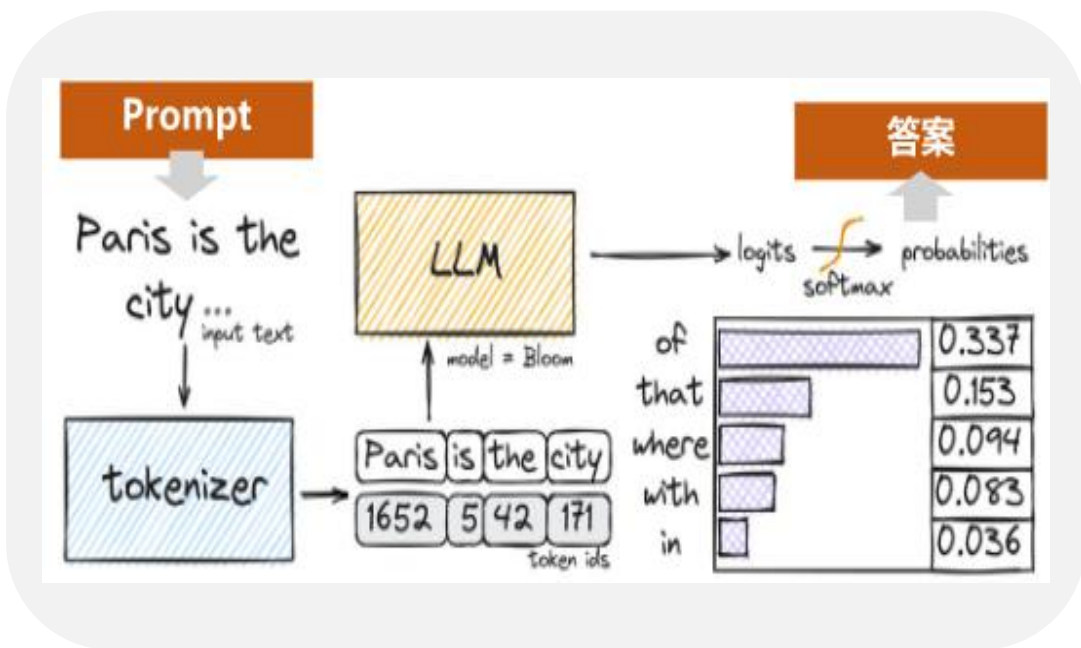
关键载体：Prompt



让大模型从“有能力”到“能做事”

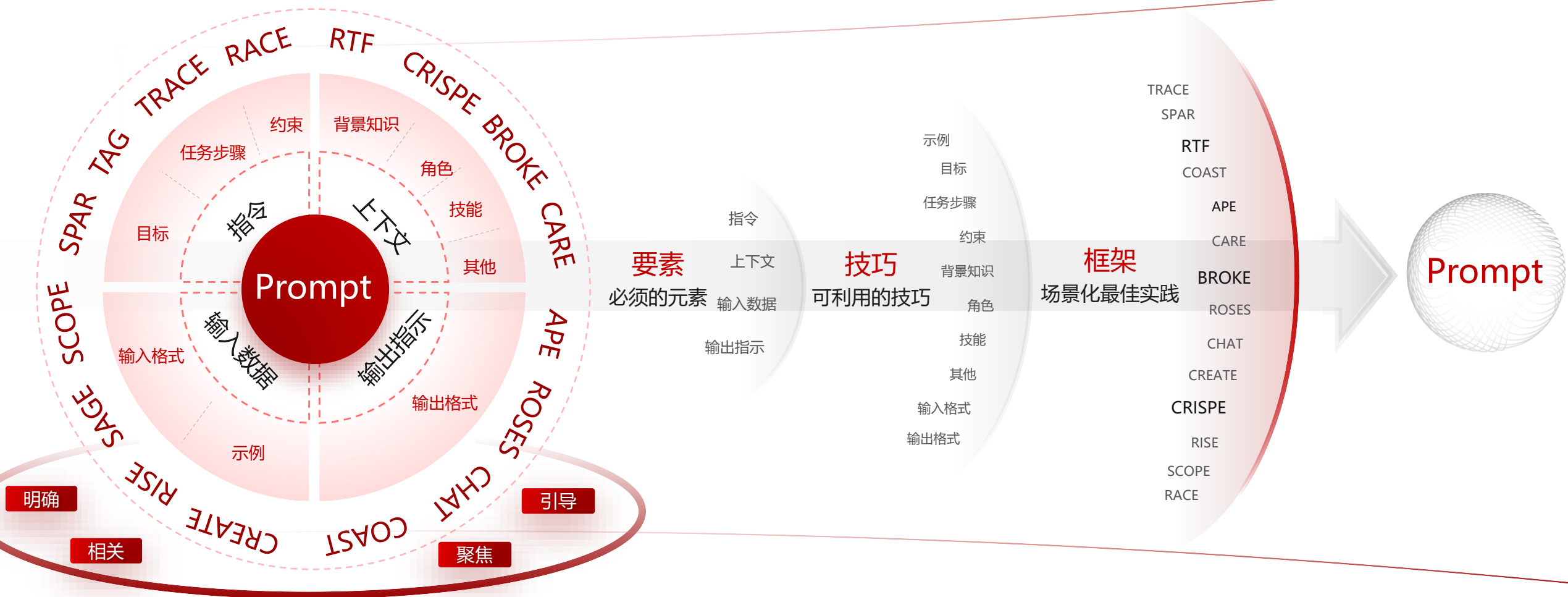
为什么需要提示词

Prompt与大模型的交互



- 提升响应质量：通过精准设计，引导模型输出更准确、更符合场景的内容。
- 增强一致性：系统提示词确保智能体在不同对话中保持一致的人设与行为风格。
- 优化交互效率：清晰的用户指令让模型快速理解任务，减少误解与冗余回答。

什么是提示词工程（ Prompt Engineering ）



提示词的设计技巧

完成任务所需步骤的详细说明：**任务步骤**

需要LLM完成的任务目标：**目标**

明确大模型的职责范围：**约束**

背景知识：介绍与任务紧密相关的背景信息

角色：让大模型扮演特定角色

技能：完成任务所需的专业能力

使模型清楚理解提示词：**使用分隔符**

说明输入的格式：**输入格式**

遵守规范和格式的高质量示例：**示例**

输入数据

指令

上下文

Prompt
技巧

输出指示

输出形式：指定输出格式，如表格、JSON

格式要求：指定响应长度、输出结构等

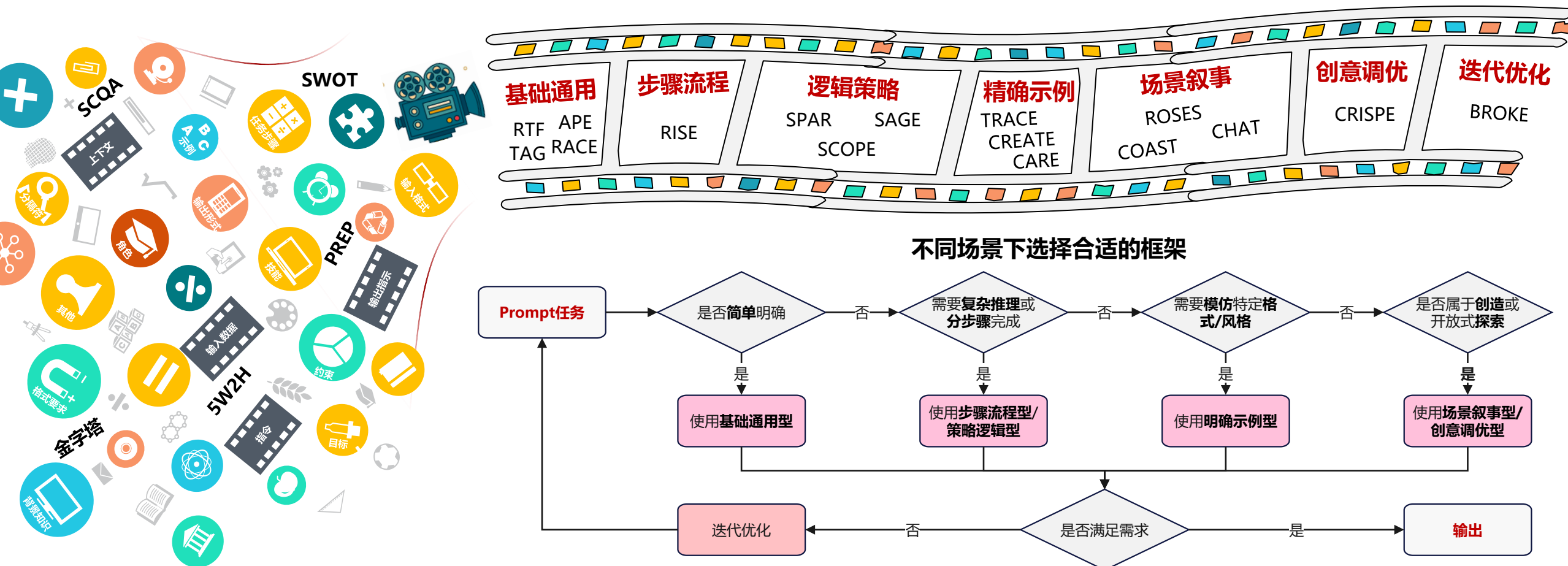
其他：未被要素包含的信息，对特定信息的补充

Prompt的设计技巧对应的理论知识

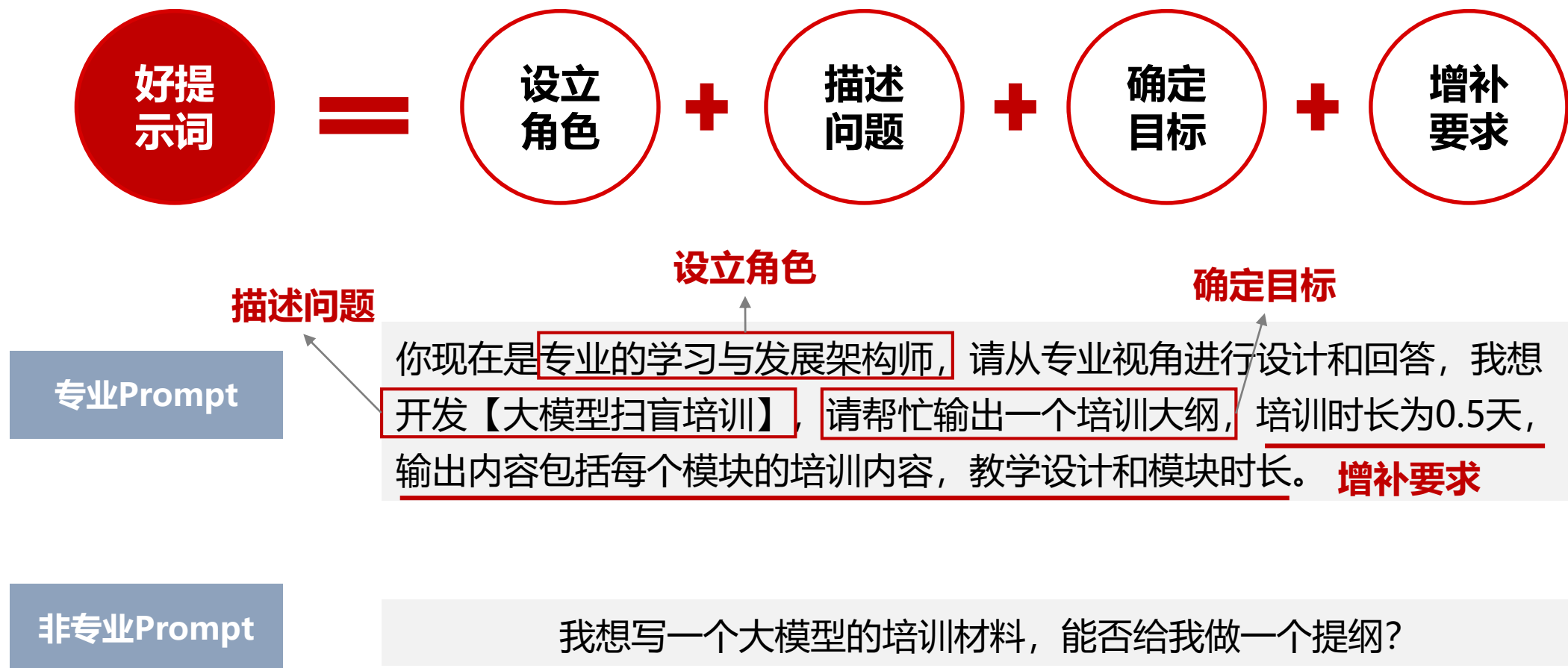
要素	技巧	理论知识	英文对照	描述
指令	任务步骤	思维链	Chain of Thought - CoT	通过生成中间推理步骤来提高 LLM 推理能力的技术。有助于 LLM 生成更准确的答案。
	设定目标	回退提示	Step-back prompting	提示 LLM 首先考虑与手头特定任务相关的一个更普遍的问题，然后将该普遍问题的答案输入到后续针对特定任务的提示中。
上下文	背景知识	上下文提示	Contextual prompting	通过提供上下文提示，可以帮助确保 AI 交互尽可能无缝和高效。模型将能够更快地理解您的请求，并能够生成更准确、更相关的响应，
	角色	角色提示	Role prompting	为生成式 AI 模型分配特定角色。这可以帮助模型 生成更相关、信息更丰富的输出，因为模型可以根据分配给它的特定角色来构建响应 。
	技能	角色提示	Role prompting	为生成式 AI 模型分配特定技能。这可以帮助模型生成更相关、信息更丰富的输出，因为模型可以根据分配给它的特定技能来构建响应 。
输入数据	使用分隔符	思维链	Chain of Thought - CoT	在思维链提示的每个示例末尾插入分隔符，帮助模型划分信息块，减少 “认知过载”
	示例	通用/零样本提示 单样本/少样本提示	One-shot & few-shot	示例可以帮助模型理解您的要求。当您 希望引导模型遵循特定的输出结构或模式时，示例尤其有用
输出指示	输出形式	自我一致性	Self-consistency	结合了采样和多数投票来生 成多样化的推理路径，并选择最一致的答案。它提高了 LLM 生成响应的准确性和连贯性。
	其他	思维树	Tree of Thoughts - ToT	使 LLM 同时探索多个不同的推理路径，ToT 从线性或独立的推理路径转向更 结构化的探索策略。允许模型在每一步考虑替代方案，评估不同的分支结果。

Prompt的设计框架

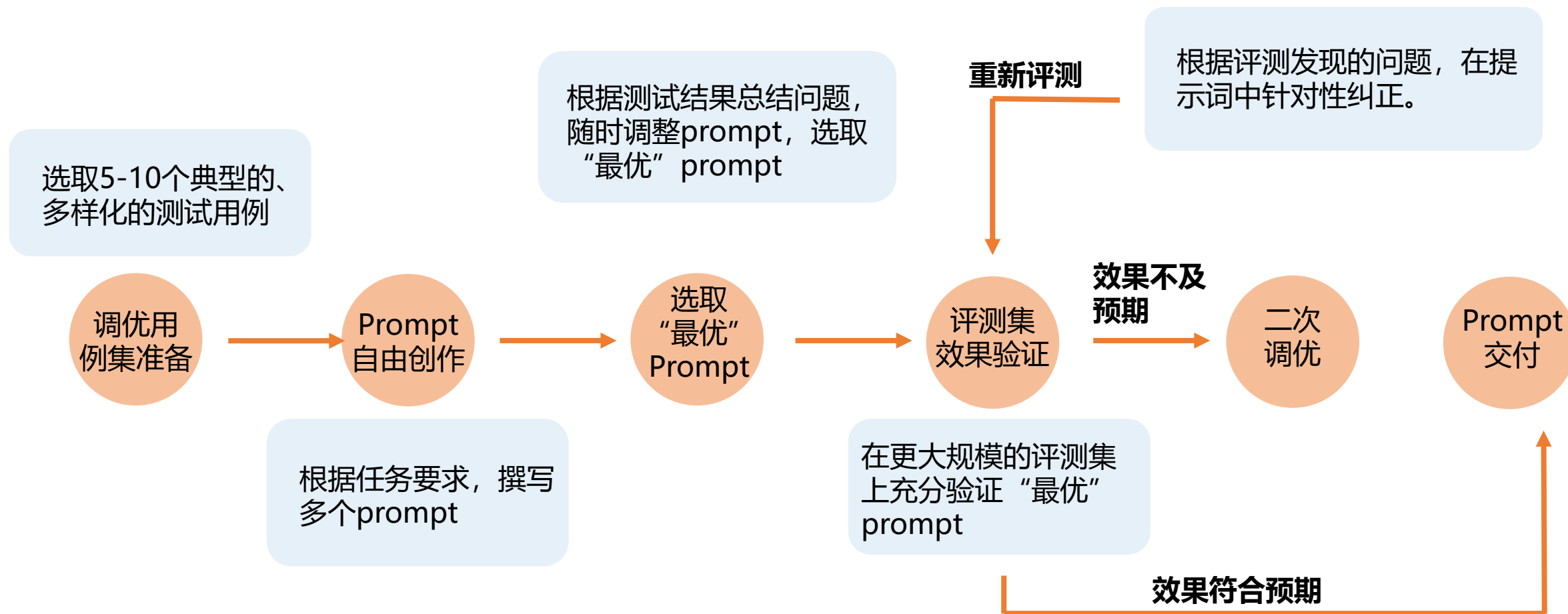
Prompt设计框架的产生，本质上是人类为了更有效地驾驭日益强大的AI模型而进行的工程化应用。它从个人经验发展为系统化的科学，并最终成为构建智能应用的底层基础设施。



简单好用的提示词框架



提示词工程调优流程



误区：期待一次性给出完美结果 ✖

误区：过度指令和模糊指令 ✖

Prompt设计原则

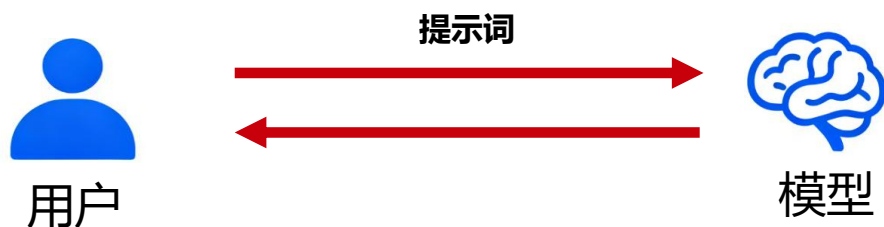
原则	说明	示例
明确目标与任务	编写提示词前，明确大模型的目标和任务，确保提示词直接对应预期的模型行为。	正例：“请描述如何保持健康的生活方式” 反例：“说点关于健康的事”
清晰性	提示词应明确表达目标，避免模糊不清。	正例：“请检查这份合同中的风险条款” 反例：“帮我看看”
准确性	提示词应基于事实，避免错误或误导性内容。	正例：“高血压患者应限制咖啡因摄入” 反例：“高血压患者可以喝咖啡”
用户友好性	提示词应使用简单、易懂的语言，避免专业术语或复杂的表达。	正例：“你是否有过疾病或对药物过敏？” 反例：“请提供你的病史和过敏史”
多样性	提示词应能够处理用户的多种表达方式，通过在提示词中提供多样化的示例或明确的风格指令，引导模型适应不同的输入变化。	“支持用户说“能推荐点地方吗？”或“有什么好玩的？””
使用上下文	在撰写提示词时，可以包含相关的上下文信息，帮助智能体或大模型理解任务背景，缩小搜索空间，减少幻觉。	“用户是30岁职场新人，预算有限，偏好自然风光”
反馈和迭代	在AgentArts的调试窗口中，根据模型的实时回答调整提示词结构，确保其符合用户需求。	收集“回答太长”“信息不全”等反馈，调整提示词的结构
遵守伦理和法律标准	确保提示词符合伦理道德和法律标准，包括但不限于保护用户的隐私，避免使用歧视性语言或行为，确保公平性和包容性。	不使用歧视性语言，不泄露用户隐私，不传播虚假信息

Agent时代下的提示词

在构建和使用智能体时，提示词分为系统提示词和用户提示词两类，系统提示词开发者在搭建智能体时就已写入，为大模型设定的初始角色设定、行为准则与响应逻辑；用户提示词即用户在使用Agent时输入的内容。

系统提示词：它决定了智能体的“人格”“风格”“边界”和整体交互方式。

用户提示词：用户在与智能体交互过程中，直接输入的具体问题或指令，用于触发特定任务或获取信息。



简单任务更“简单”



复杂任务从指挥官变成规则制定者

Prompt输入参数

参数类别	参数	参数类型	取值范围	说明
通用	model	string	-	用于生成内容的模型ID。
	messages	object[]	-	对话组成的消息列表。不同模型支持不同类型的消息。
系统提示词	messages.role	string	system	发送消息的角色。
	messages.content	string/object[]	-	当前角色发出的消息内容。
用户提示词	messages.role	string	user	发送消息的角色。
	messages.content	string/object[]	-	当前角色发出的消息内容。
	messages.content.type	string	text	消息类型。

大模型输出参数

大语言模型预测下一个Token可能是什么的概率时，词汇表中的每个Token都会获得一个概率。然后对这些Token概率进行采样，以确定将生成的下一个Token。温度（Temperature）、Top-K 和 Top-P 是最常见的设置，它们决定了如何处理预测的Token概率以选择单个输出Token。



Temperature



Top K



Top P

模型输出控制参数

参数类别	参数	参数类型	取值范围	说明
模型输出行为配置	max_tokens	integer/null	各个模型不同	模型回答最大长度，单位为Token。
	temperature	float/null	[0, 2]	采样温度，用于控制模型生成内容的随机性。
	top_p	float/null	(0,1]	核采样概率阈值，用于控制模型生成内容的多样性，和temperature参数类似，但原理不同，可以更精细地控制模型输出的词汇范围。
	top_k	integer	≥0	控制模型生成时每次只从概率最高的k个词里挑选，用来控制生成文本的随机性。



目录

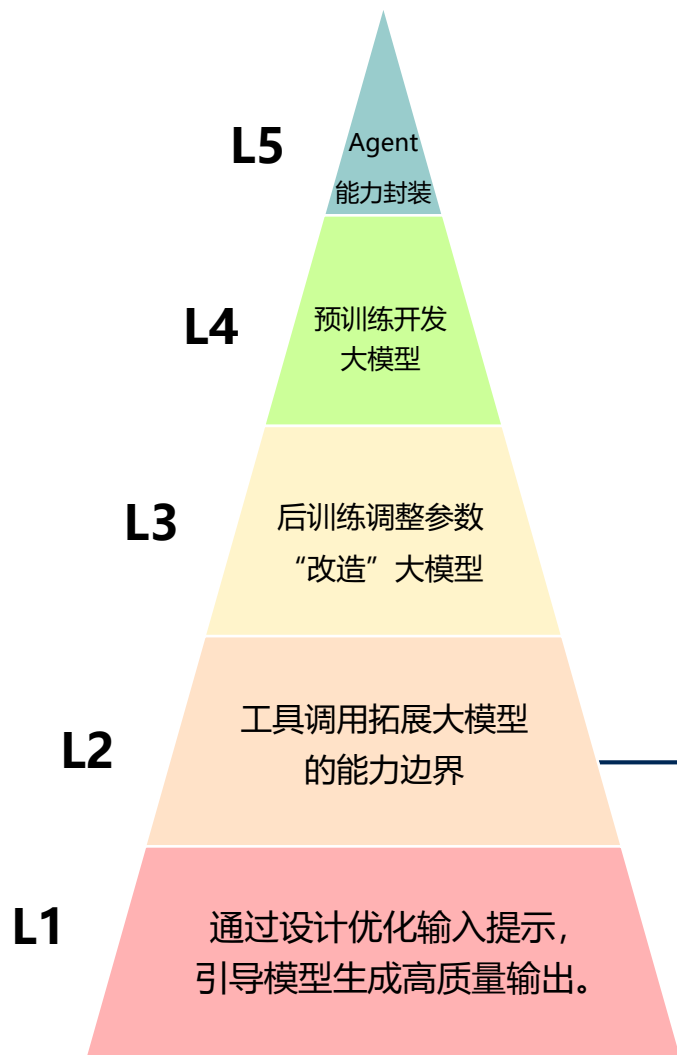
1. 大模型应用技术发展

2. Prompt工程

3. 大模型的局限与工具调用

- **2.1 RAG**
- 2.2 Plugin与Tools
- 2.3 Function Calling与MCP
- 2.4 Skills

大模型的应用落地的五级阶梯



2、工具调用：指大模型应用在接收到用户指令后，不直接生成最终的自然语言回复，而是输出一段具备明确语法结构的调用指令，经由外部执行引擎解析后，去调用预定义的知识库、第三方函数、API接口或本地插件，并将获取的执行结果回填至模型上下文，最终生成满足用户需求的答复。

什么是RAG

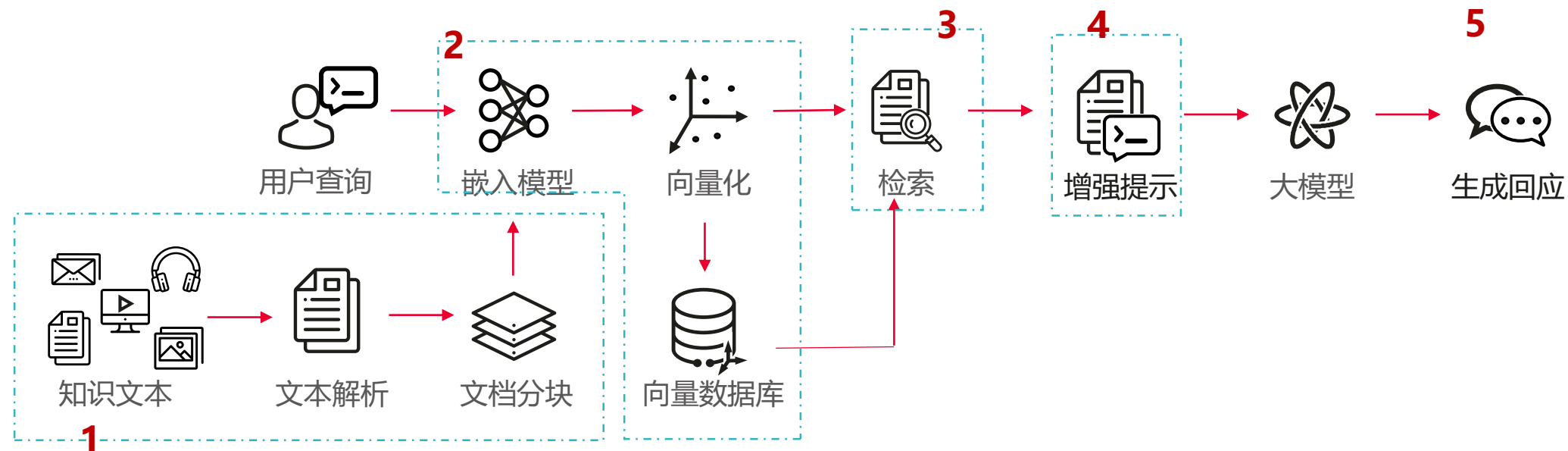
RAG (Retrieval-Augmented Generation, 检索增强生成) 结合了信息检索和文本生成技术。使大型语言模型通过从数据源检索信息来生成回答，提高回答的准确性和上下文相关性。



为什么需要RAG?



RAG的基础架构



文档预处理

1. 将原始文档切分成语义完整的文本块，并可选地清洗格式、去除噪声。

向量化

2. 将文本块转换为高维数值向量，便于计算语义相似度。

向量检索

3. 从向量库中召回相似文档，对结果按相关性精准重排。

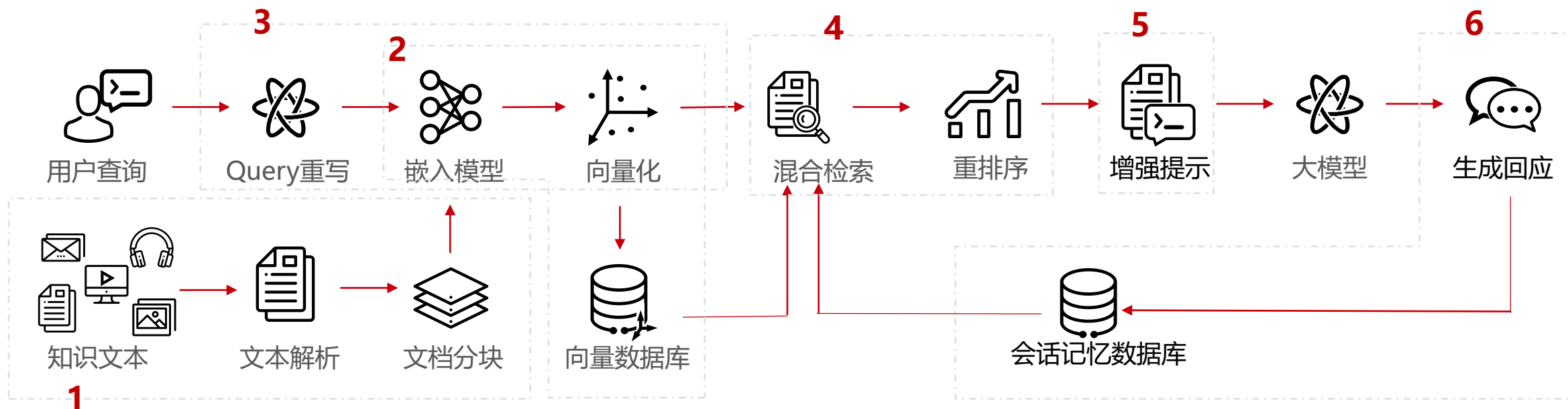
增强提示

4. 将检索到的文本块与用户问题拼接成包含上下文的提示模板。

大模型回应

5. 基于增强后的提示生成准确、可溯源的最终答案。

RAG的优化架构



文档预处理

1. 将原始文档切分成语义完整的文本块，并可选地清洗格式、去除噪声。

向量化

2. 将文本块转换为高维数值向量，便于计算语义相似度。

Query重写

3. 对用户原始提问进行优化或分解，使其更适合检索。

检索与重排序

4. 从不同检索方式中召回相似文档，对结果按相关性精准重排。

增强提示

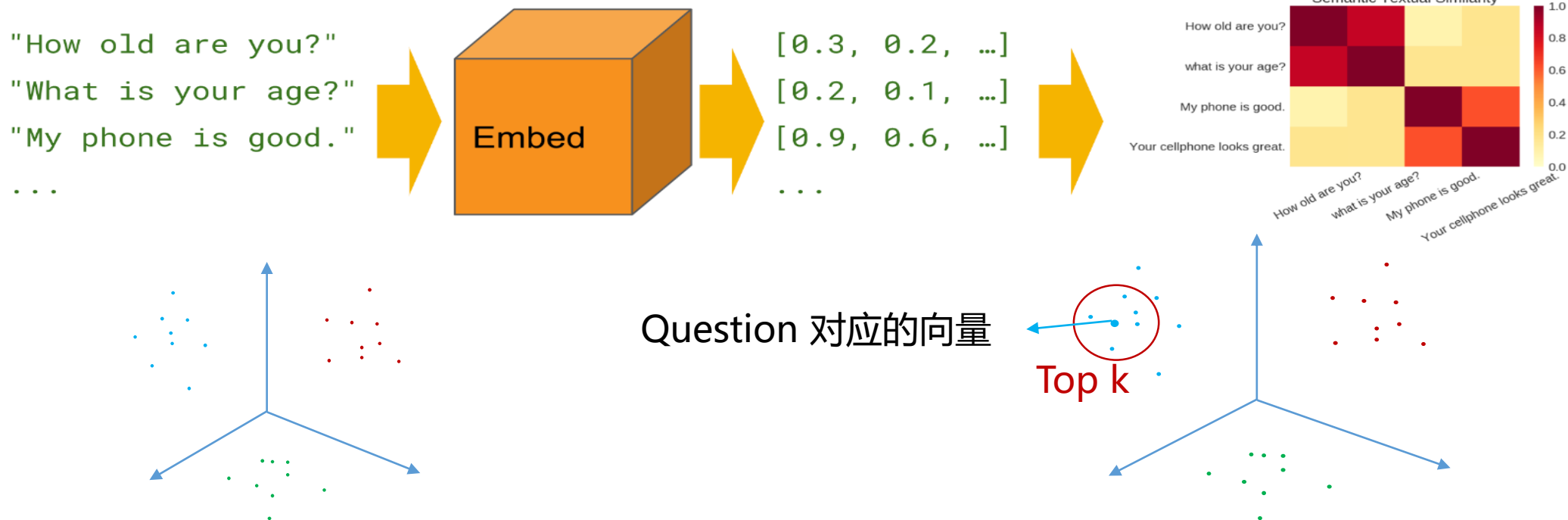
5. 将检索到的文本块与用户问题拼接成包含上下文的提示模板。

大模型回应

6. 基于增强后的提示生成准确、可溯源的最终答案。

向量化

向量化 (Embedding) 是将离散数据转换为连续向量的技术，作用是通过向量空间中的距离度量语义相似度，从而让机器能够理解和检索内容的含义。



检索与重排序

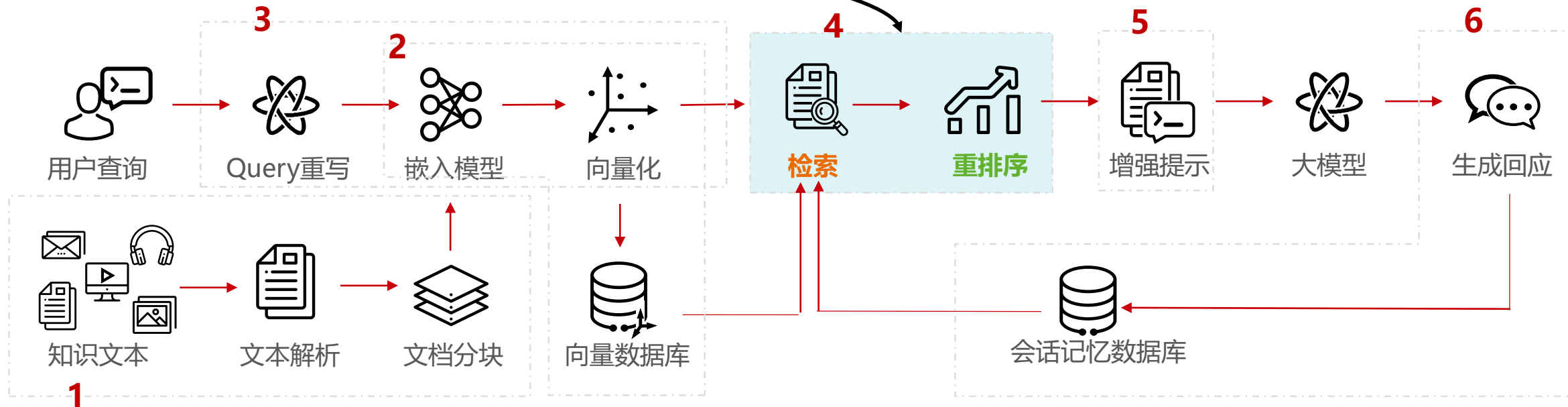
检索与排序与图书管理员类似



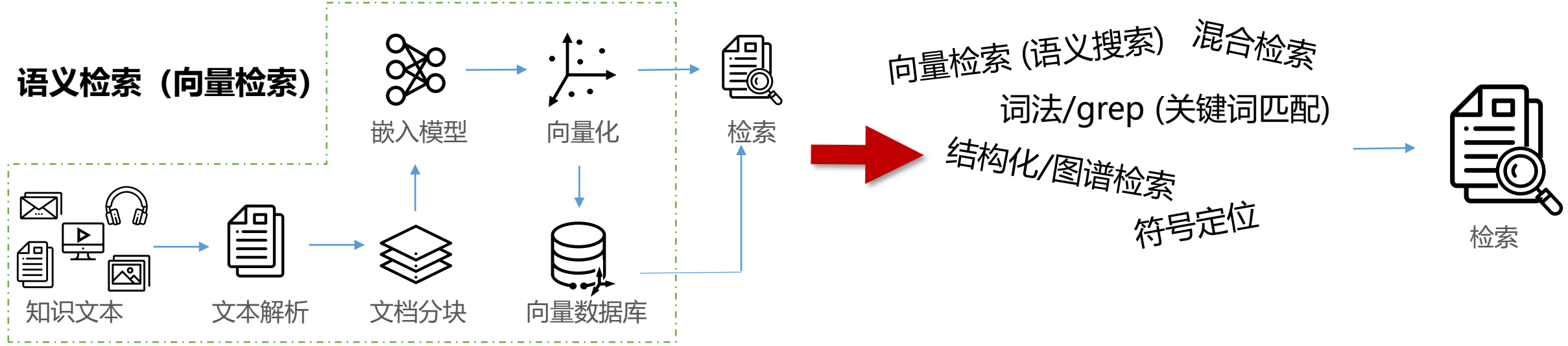
- 1、理解用户的含义
- 2、找到相关的内容书籍
- 3、给用户推荐最合适的内容

检索与排序

- 1、文档的**相关性**：返回相关文档并排除无关文档
- 2、文档返回的**数量**：大量无关文档。浪费上下文窗口
- 3、只返回**排名最高**的文档：可能会错过有价值的信息
- 4、**监控与实验**：没有完美的解决方案，在业务系统中通过监控与实验调整设置以找到有效的方法

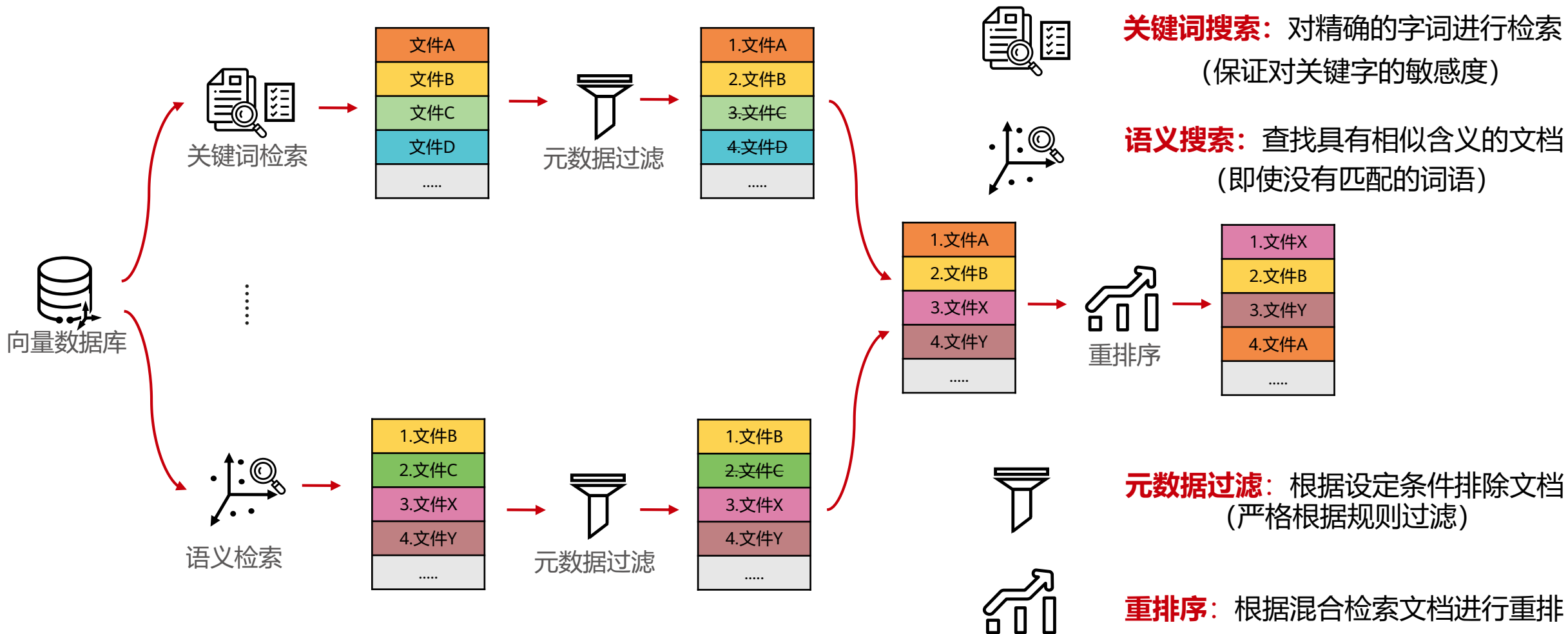


RAG中的检索技术



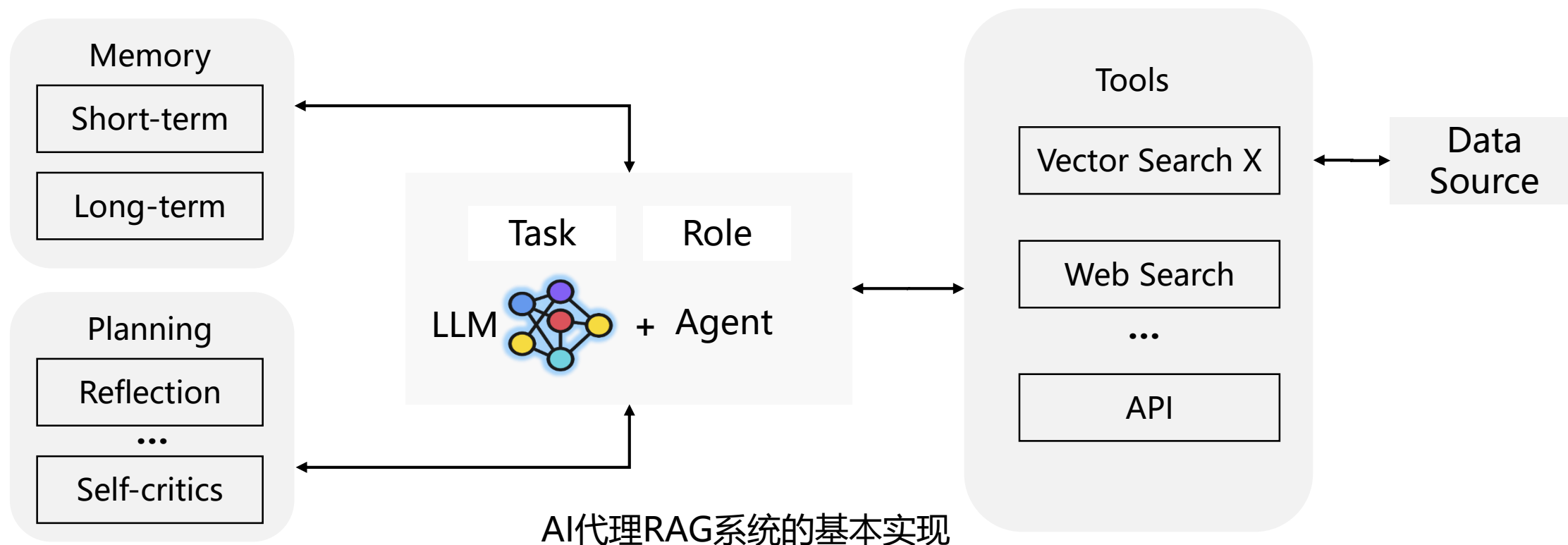
检索技术	在RAG中的角色	应用场景
词法/grep (关键词匹配)	作为 稀疏检索器 ，通常用BM25算法实现。	检索精确的标识符、字符串常量；作为混合检索中的“精确匹配”部分。
向量检索 (语义搜索)	作为 稠密检索器 ，利用嵌入模型和向量数据库。	理解自然语言查询意图，匹配语义相似的代码片段。
结构化/图谱检索	作为 知识图谱增强的检索器 ，预先抽取代码结构关系。	检索函数调用链、类继承关系、数据流等结构化信息，回答“影响哪些地方”等问题。
符号定位	作为 快速索引检索器 ，类似IDE的定义跳转。	快速定位符号定义位置，作为RAG的预处理步骤或精确回退。
混合检索	作为 多路检索融合器 ，结合稀疏和稠密检索结果，再通过重排序（Reranker）优化。	大多数生产级RAG系统（如Cursor、Continue）的实际方案，兼顾精确和语义。

混合检索



Agentic RAG

Agentic RAG（基于智能代理的检索增强生成）是传统 RAG 的增强版本，它通过引入 AI Agent（人工智能代理）的动态规划、自主决策和多步推理能力，使 RAG 系统能够更智能地处理复杂查询任务。





目录

1. 大模型应用技术发展

2. Prompt工程

3. 大模型的局限与工具调用

- 2.1 RAG
- **2.2 Plugin与Tools**
- 2.3 Function Calling与MCP
- 2.4 Skills

为什么需要插件与工具？

大语言模型无法突破对话框的限制进行自主行动。工具调用是扩展大模型能力的关键技术，使模型能够超越自身的限制，与外部世界交互并执行复杂任务。



扩展能力边界

突破模型限制，赋予Agent与外部系统交互的能力



实时信息获取

获取最新数据和实时信息，确保决策基于最新上下文



执行复杂操作

执行多步骤任务流，实现从信息获取到操作执行的闭环

什么是插件、工具？

工具（Tool）：是插件内具体的执行单元，每个工具对应一个独立的功能，负责完成一项具体的单一任务。

插件（plugin）：是大语言模型（LLM）与外部世界交互的一系列工具，是集成了一个或者多个工具（Tools）的功能集合，它定义了这些工具共用的基础属性（如服务域名、鉴权方式）。

插件名称	工具名称	API接口地址
文字识别插件	通用文字识别	https://{endpoint}/v2/{project_id}/ocr/ general-text
	通用表格识别	https://{endpoint}/v2/{project_id}/ocr/ general-table
	手写文字识别	https://{endpoint}/v2/{project_id}/ocr/ handwriting

常见的插件形式

API形态插件：API类型插件是将现有的RESTful API（HTTP/HTTPS接口）封装为应用可调用的工具。它本身不承载业务逻辑，而是作为桥梁将指令“翻译”为API请求，交由外部系统执行逻辑并返回结果。

JSON文件形式导入API插件：可以直接将API插件中定义的接口、入参、出参、鉴权等信息编辑为一份JSON文件，再导入应用创建API形态的插件。





目录

1. 大模型应用技术发展

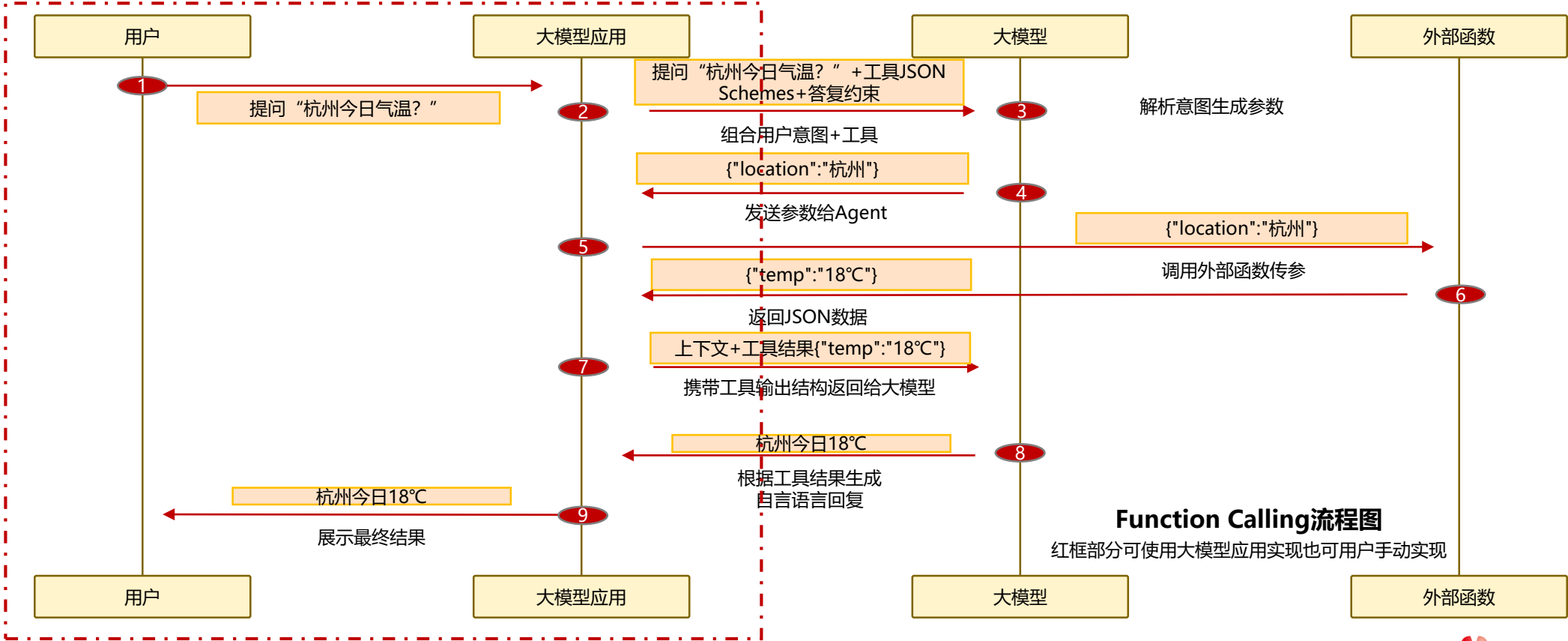
2. Prompt工程

3. 大模型的局限与工具调用

- 2.1 RAG
- 2.2 Plugin与Tools
- **2.3 Function Calling与MCP**
- 2.4 Skills

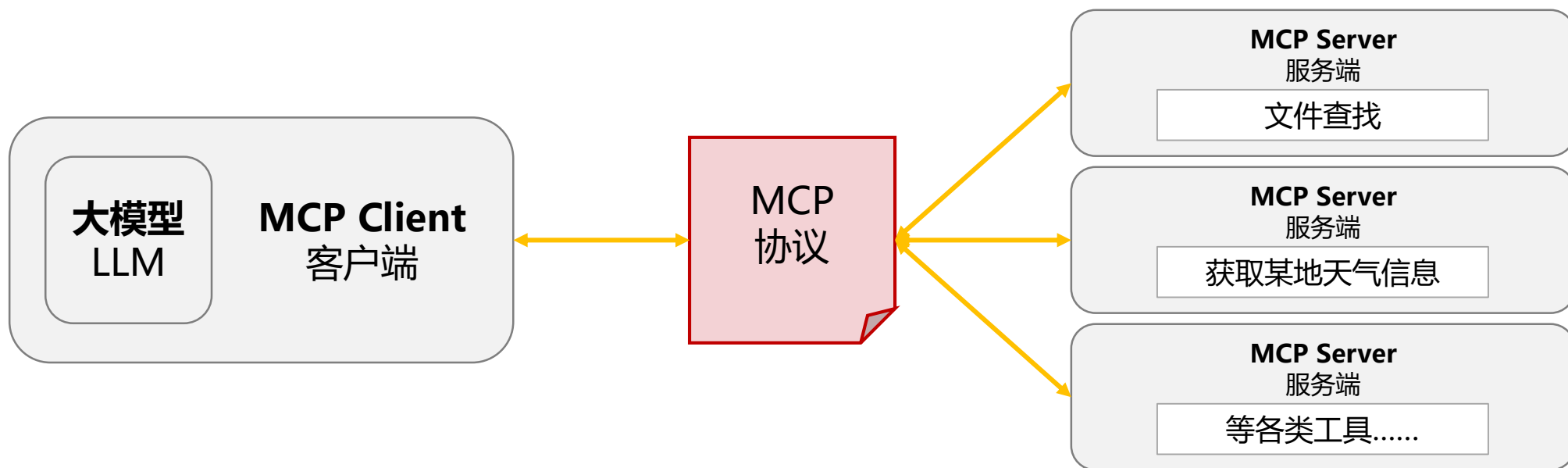
什么是Function Calling

Function Calling指大模型调用预定义函数的能力，允许模型生成请求参数并由外部系统执行具体操作，将结果反馈给模型，由模型整合信息进行回应。



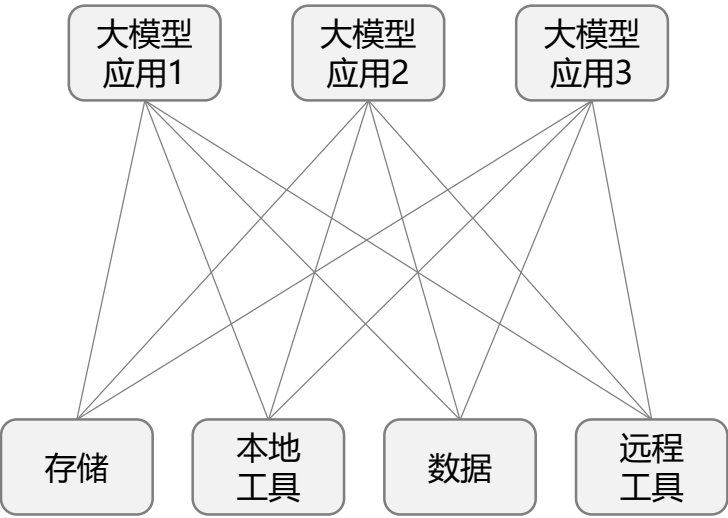
什么是MCP

MCP（Model Context Protocol，模型上下文协议）是Anthropic公司在2024年底主导发布的一个开放的、通用的、有共识的协议标准。MCP被设计为通用接口，MCP协议使大型语言模型（LLM）能标准化接入外部数据源与工具。

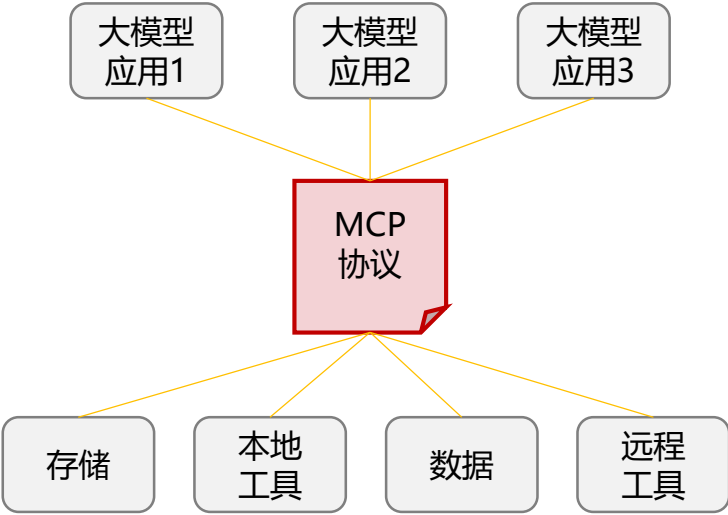


为什么需要MCP?

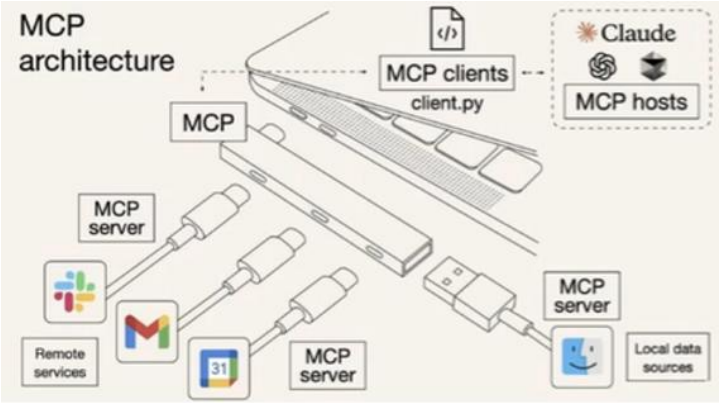
Function Calling面向大量的工具及模型应用场景需要逐一适配，造成大量重复劳动。而MCP则为大模型与外部数据和工具的无缝集成提供了标准化通信协议。



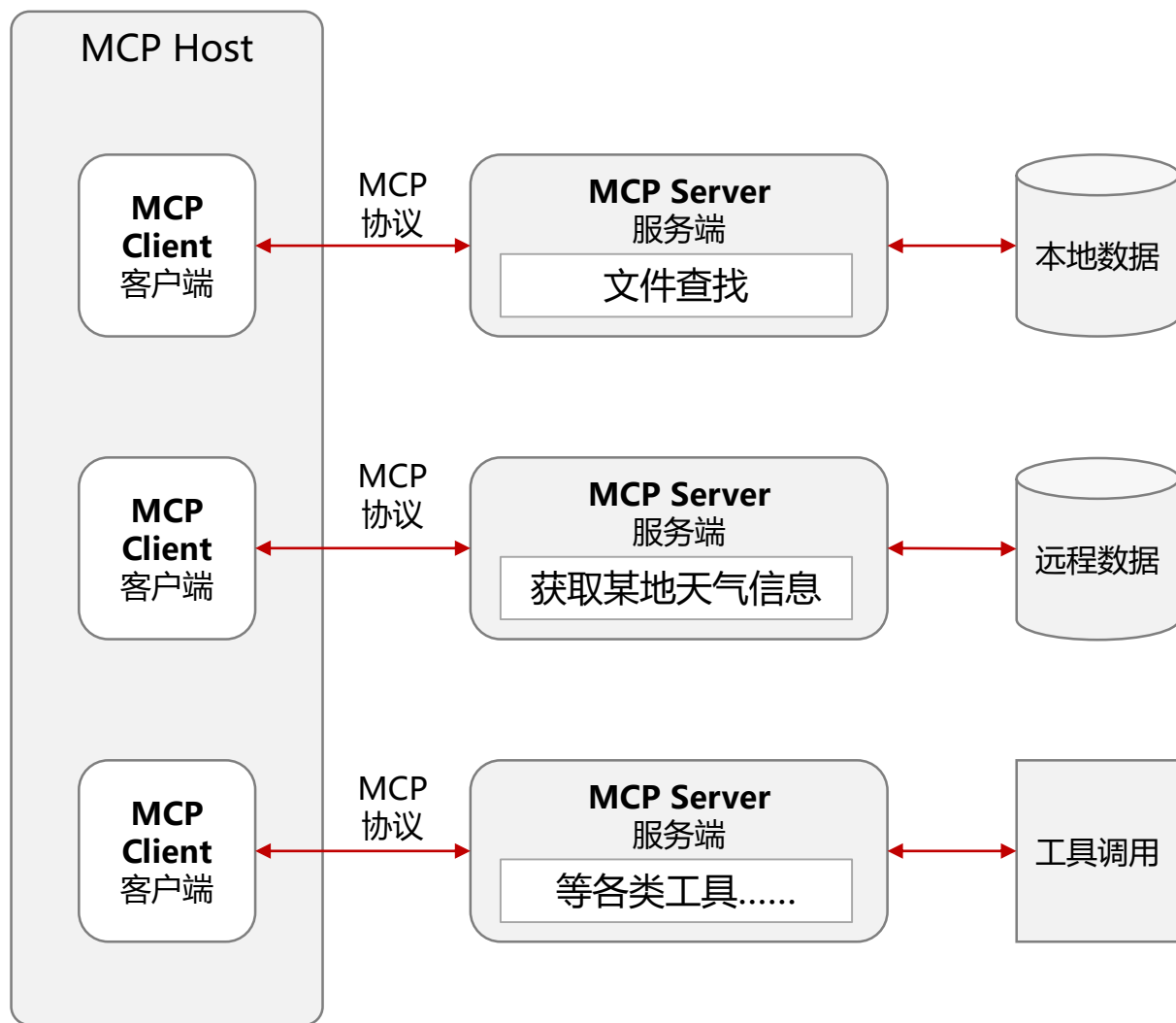
无MCP



有MCP



MCP内部架构

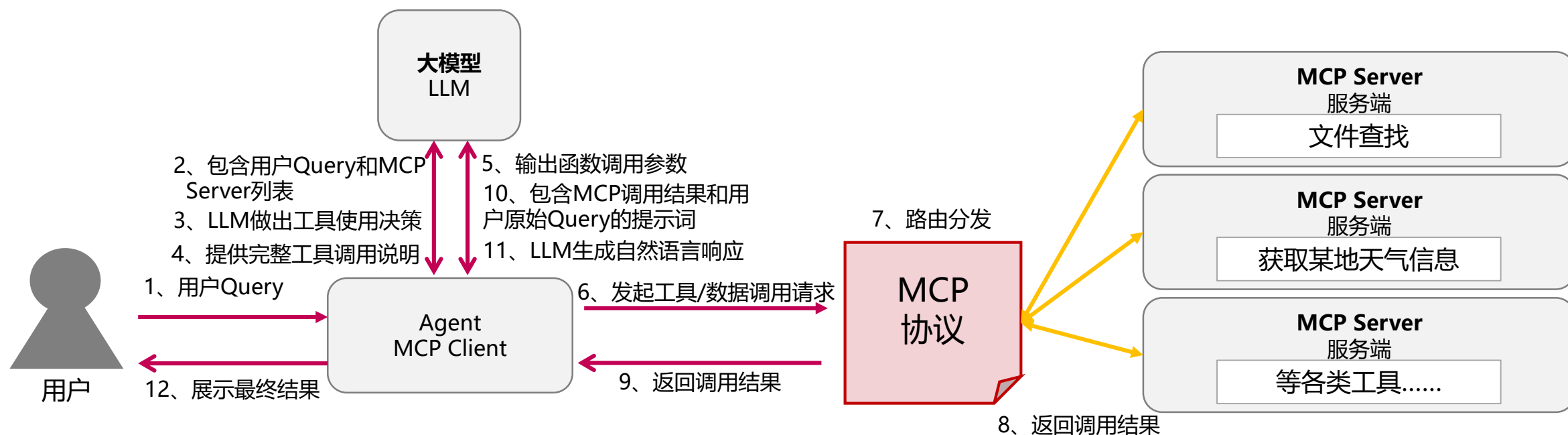


• MCP 遵循CS架构，其中：

- **主机 (MCP Hosts)**：大模型应用程序发起连接，通过MCP协议访问或调用工具。
- **客户端 (MCP Clients)**：在主机应用程序内，与服务器保持 1:1 连接。
- **服务器 (MCP Servers)**：轻量应用程序为客户端提供上下文、工具和提示信息。

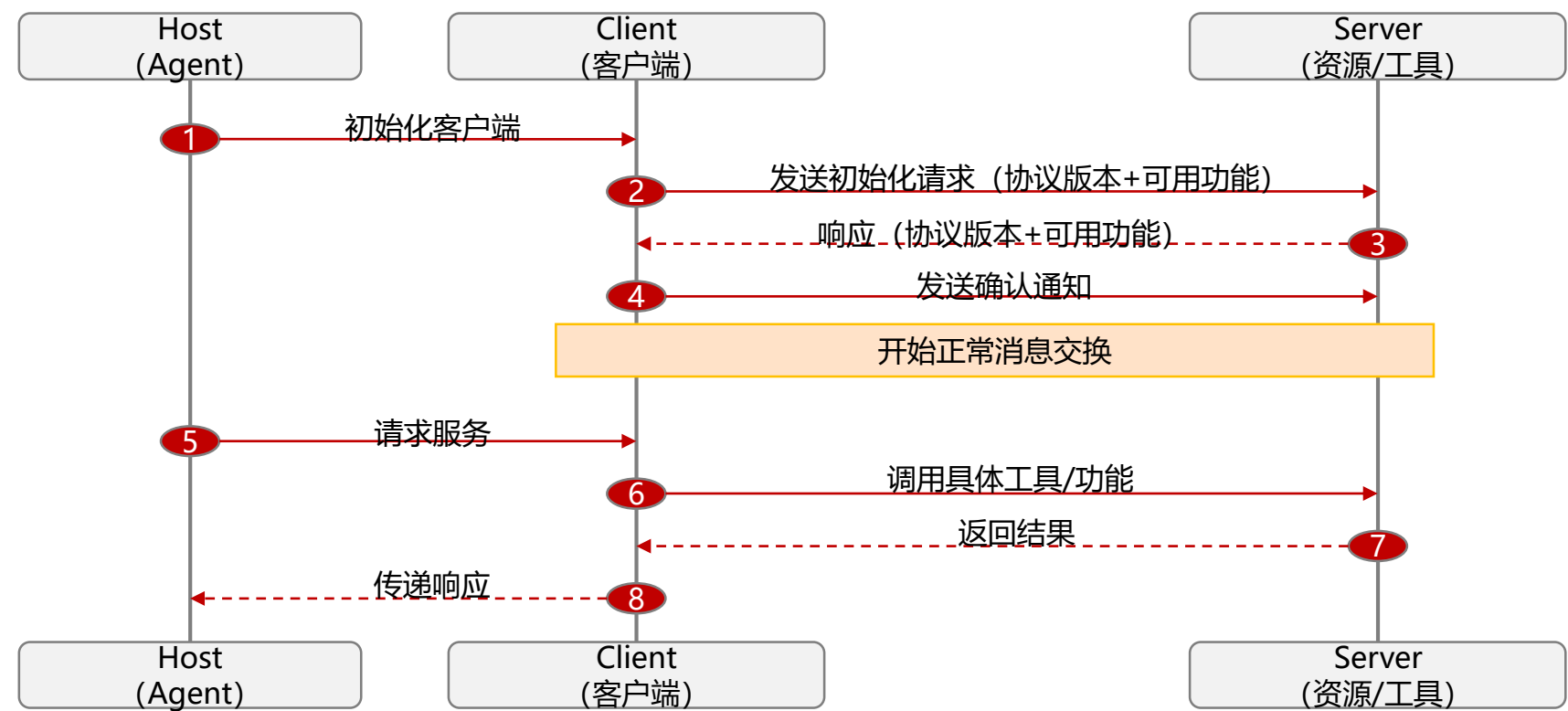
MCP与大模型/大模型应用交互的流程

MCP协议作为一种开放的标准化协议，规范了Agent如何与外部工具和数据进行交互。模型通过MCP Client与MCP Server进行交互，MCP Server可以由第三方开源社区提供或自行开发。整体流程如下图：



用MCP连接及协同外部服务的典型 workflows

大模型应用需要先初始化客户端，确认服务状态，在调用时根据用户问题选择可用的外部服务资源，通过客户端连接服务端调用具体的服务。



MCP与Function Calling对比

对比维度	优劣小结
架构层级	FC 架构轻量化、无额外部署成本；MCP 引入中间层，架构更规范但部署工作量更高
Token 开销	少量工具场景 FC 开销更低；大批量工具接入时 MCP 显著节约 Token 消耗
多模型适配开发	单模型小项目 FC 开发效率更快；多模型、多工具集群场景 MCP 大幅减少重复开发工作
调用链路与时延	FC 链路短、调用延迟更低；MCP 标准化路由，延迟小幅增加但工具统一调度能力更强
部署拓展能力	简易本地 Demo 优先 FC；跨地域、分布式复杂业务场景更适配 MCP
安全与运维管控	小项目 FC 无需冗余安全配置；企业级合规场景 MCP 运维、安全优势突出
适用落地场景	小体轻量级项目优选 FC；中大型规模化工程落地优选 MCP
二者协作定位	实际工程普遍采用 FC 生成调用指令 + MCP 统一调度工具的组合方案

MCP市场

随着大模型应用的发展，企业 AI 落地需要依靠大量的MCP服务打通系统底层通路连接各种工具，因此应运而生了MCP市场，满足开发者快速检索下载MCP服务的需求。

海外

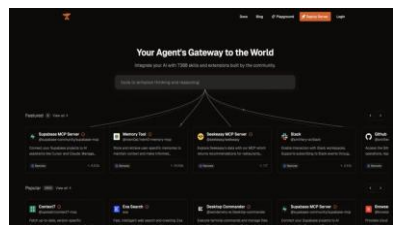
<https://mcp.so/>

全球存量 MCP 资源最全，一键复制部署指令，兼顾中英文检索，全品类通用 MCP 首选检索站



<https://smithery.ai/>

支持远程托管 + 本地双部署，分类精细，一键拉取运行，个人开发者快速落地 MCP 首选



国内

<https://mcpmarket.cn/>

国内原生中文 MCP 聚合市场，本土化软件适配完善、配套中文教程社群，新手入门优选



<https://www.mcp-cn.com/>

精选严控质量，过滤劣质 MCP，聚焦国内 SaaS / 云服务，轻量化商用采购





目录

1. 大模型应用技术发展

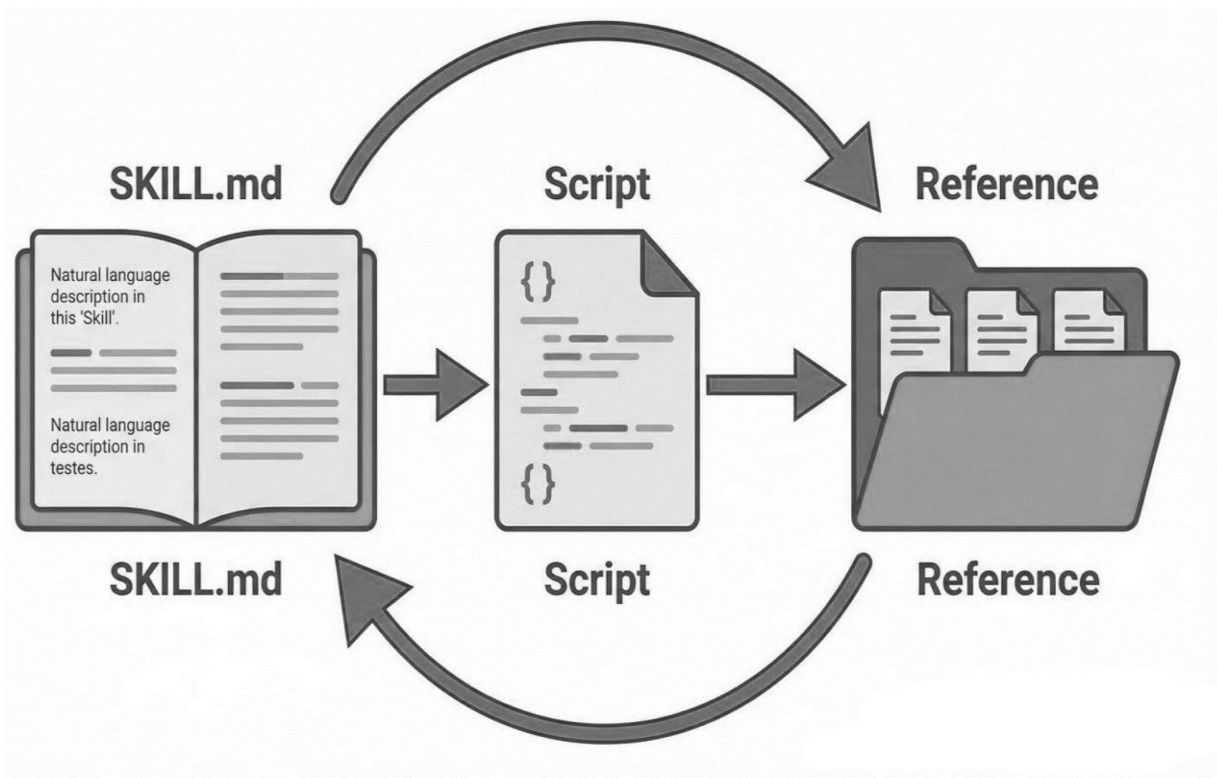
2. Prompt工程

3. 大模型的局限与工具调用

- 2.1 RAG
- 2.2 Plugin与Tools
- 2.3 Function Calling与MCP
- **2.4 Skills**

什么是Skill?

Skill是一个被标准化封装的能力单元，能像函数一样反复使用。一个 Skill 就是一个文件夹，通过按需、渐进式加载，实现高效且可复用的经验传递。



Skill 可以包含这几个部分：

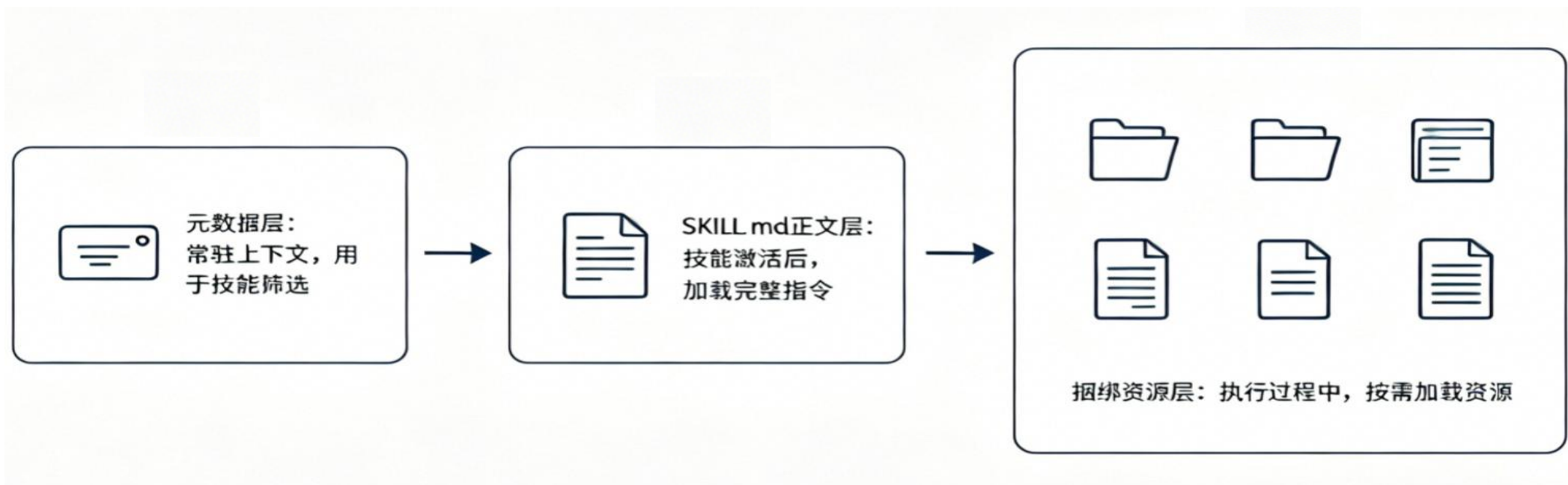
- **SKILL.md**：这是核心说明文件，告诉 AI 这个 skill 能做什么、怎么用
- **scripts 目录**：放一些可执行的脚本，比如 Python 分析工具
- **references 目录**：存放参考资料和最佳实践文档
- **assets 目录**：各种模板和资源文件

解剖 Agent Skill：标准目录结构



为什么要使用Skill?

Skill采取渐进式加载的三层架构，初始仅加载元数据层及正文层中的Name、Description字段，减少Token消耗及上下文窗口。



SKILL目录结构

Skill本质上是一个文件夹，一个 skill 通常不是单个文件，而是一个独立目录。以本地预设的 **skill-creator** 为例，它的大致结构如下：

skill-creator/ ├── SKILL.md	//负责描述什么时候使用这个 skill，以及真正的操作流程。
├── agents/ │ ├── openai.yaml	//放产品或界面侧需要读取的元数据。
├── assets/ │ ├── skill-creator-small.svg │ └── skill-creator.png	//放图标、模板、图片、字体等资源。
├── references/ │ └── openai_yaml.md	//放补充文档，在需要时再加载。
├── scripts/ │ ├── generate_openai_yaml.py │ ├── init_skill.py │ └── quick_validate.py	//放可执行脚本，用来处理重复、确定性强、容易出错的步骤。
└── license.txt	

SKILL.md的组成结构

name (技能标识符)

- Skill的唯一ID，也是其在系统和日志里被引用的方式。命名原则很简单——动词+名词。

名词+命名：

查询天气

description (技能描述)

- 应用是否激活Skill判断的唯一依据，避免激活判断模糊。

技能描述：

当用户询问天气情况时，激活此技能

执行指令

- Skill激活后具体执行剧本，包含结构化的步骤说明、分析框架、输出格式要求、边界条件等

步骤说明：

步骤1：调用查询天气MCP

步骤2：解析查询结果

步骤3：按天气预报的形式回复

SKILL与MCP的对比

MCP 负责打通外部资源的连接通路、全量预加载上下文；Skill负责按需封装业务调用逻辑、惰性按需加载，后者大幅压缩 Token 开销、降低使用门槛。

	 MCP (Model Context Protocol)	 Agent Skills
核心价值	 标准化访问接口	 程序性知识封装
抽象层级	 Transport Layer 传输层/连接层	 Application Layer 应用层/逻辑层
解决问题	 How to Connect 如何连接外部资源	 How to Use 如何有效使用这些资源
技术形式	 JSON-RPC 2.0 协议 { "jsonrpc": "2.0", "method": "..." }	 Markdown + Scripts 自然语言+脚本
上下文策略	 Eager Loading 急切加载（一次性全部）	 Lazy Loading 惰性加载（渐进式按需）
目标用户	 Developers 开发者	 Everyone 所有人（开发者+领域专家+业务人员）
 互补关系，共同构建完整的智能体能力体系		
Token效率对比： MCP: 16,000 tokens → Skills: 500 tokens (节省96%) ↑		

什么时候使用Skills

当需要降低 Token 消耗、面向非开发人员落地能力、按需动态调用工具、轻量化封装业务逻辑时，优先选用 Agent Skills；反之底层链路对接、标准化协议互联场景用 MCP。

- 1, 发现自己在向 AI 反复解释同一件事
- 2, 某些任务需要特定知识、模板、材料才能做好
- 3, 发现一个任务要多个流程协同完成

1. “帮我分析数据”、
“要把异常数据先处理掉”、
“数据不要使用平均数。。。 ”
2. “帮我分析需求，进行需求开发”、
“注意代码结构问题”、
“常量要统一放在xxx”。。。
- 3.任务需要多个步骤协同，比如
需求设计 ->
代码开发 ->

Skills市场

与MCP市场一样，企业 AI 落地需要大量的 Skill 以便于低成本落地行业业务能力，随着市面上 Skill 数量迅猛增长，Skills 市场也开始出现。

海外

<https://github.com/ComposioHQ/awesome-claude-skills>

Claude 领域权威人工精选清单，技能质量高、更新稳定，新手快速获取优质成熟 Skill

<https://skillsmp.com/zh>

全球海量开源 Claude 技能一站式聚合，智能检索 + 质量筛选，找零散小众 Skill 首选

国内

<https://github.com/heilcheng/awesome-agent-skills/blob/main/README.zh-CN.md>

中文整理全品类 Agent 技能导航，一站式汇总优质开源仓库，适合系统性学习选型

<https://skillhub.cn/>

专为国内用户本土化优化，中文检索 + 国内高速下载，适配国产软件与本土 AI 框架。

常用Skill推荐

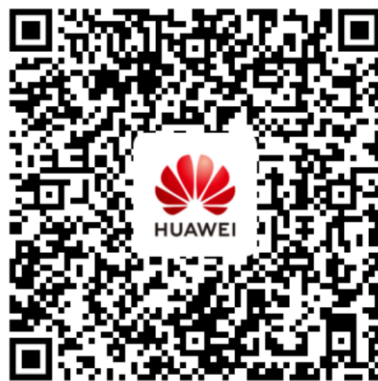
- 1、 skill-creator: 一个帮助开发者安装skill的skill
 - <https://github.com/anthropics/skills/tree/main/skills/skill-creator>
- 2、 find -skill: 一句话帮你找到想要的skill
 - <https://skills.sh/vercel-labs/skills/find-skills>
- 3、 Anthropic开源的17款Skills, 其中有office四件套docx、 pptx、 xlsx、 pdf, 以及MCP Builder则主要用来写mcp协议。
 - <https://github.com/anthropics/skills>



学习推荐

- 华为云官方网站

- 华为云官网: <https://www.huaweicloud.com/>
- 华为云学堂: <https://edu.huaweicloud.com/>



华为云学堂

感谢

版权所有©2026，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编、也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。